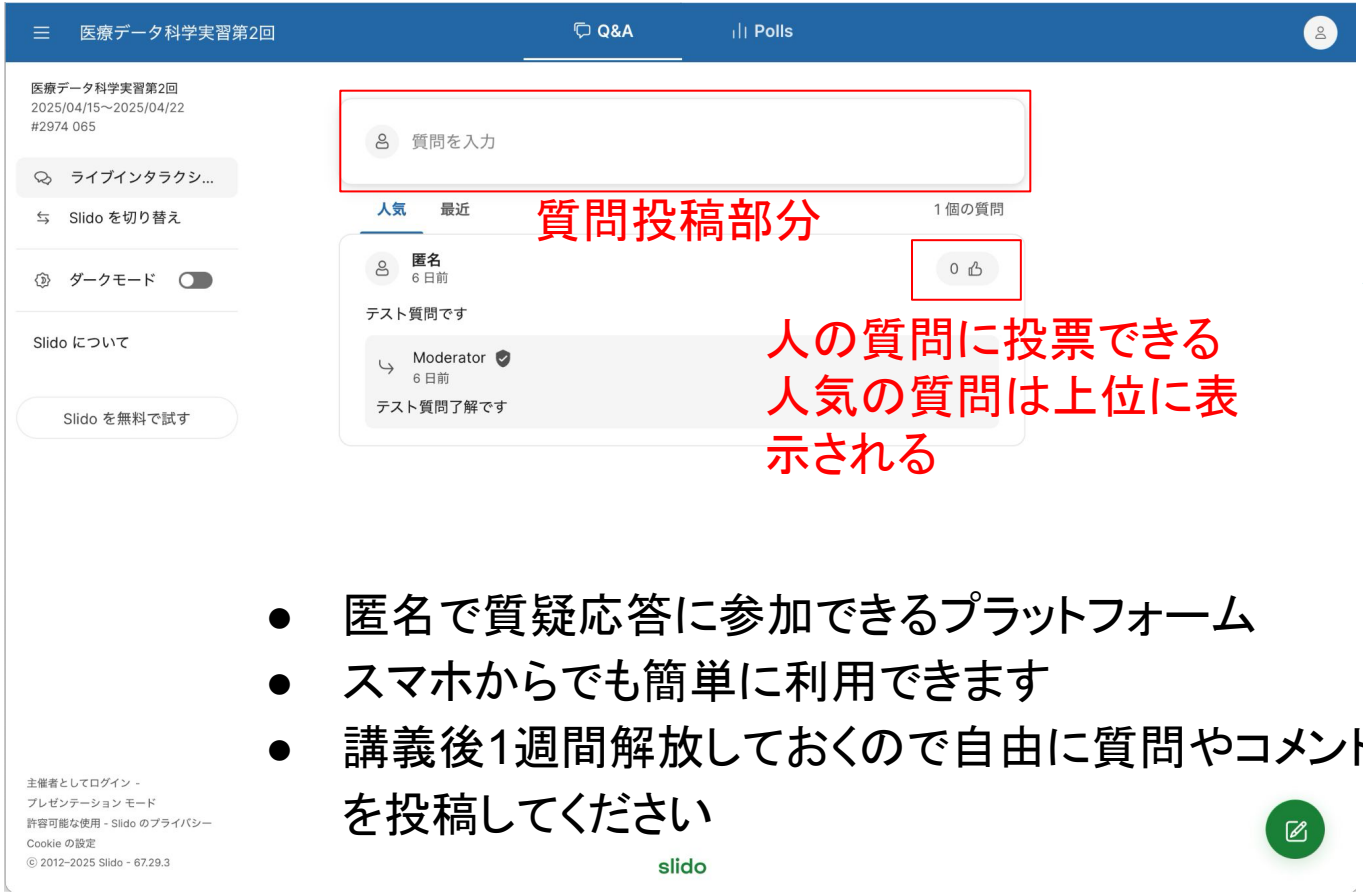
簡単な計算、ベクトル、行列とそれらの計算

医療データ科学実習
（Practice of Biomedical Data Science）
第2回　2026年4月21日

医療データ科学分野
matsui.kota.8k@kyoto-u.ac.jp

Slidoで質疑応答に参加しよう



参加URL&QRコード
https://app.sli.do/event/bTkgBJ7tMKyfAx8UDoYLwh

- 匿名で質疑応答に参加できるプラットフォーム
- スマホからでも簡単に利用できます
- 講義後1週間解放しておくので自由に質問やコメントを投稿してください

Chap1. R言語による計算

- インデックス操作
- ベクトル演算
- 行列演算

Chap1. R言語による計算

- インデックス操作
- ベクトル演算
- 行列演算

Rにおけるベクトルとインデックス

- Rによるベクトル（1次元配列）は `c()` 関数または `:` 演算子などで作成できる

例1：

```
> x <- c(10, 20, 30, 40, 50)
> x
[1] 10 20 30 40 50
```

$x = (10, 20, 30, 40, 50)$

というベクトルを作った

例2：

```
> y <- 1:10
> y
[1] 1  2  3  4  5  6  7  8 9
10
```

$y = (1, 2, 3, 4, 5, 6, 7, 8, 9, 10)$

というベクトルを作った

例3：

```
> fruits <- c("apple", "banana", "cherry", "date")
> fruits
[1] "apple"  "banana" "cherry" "date"
```

$\mathrm{fruits} = (\mathrm{apple}, \mathrm{banana}, \mathrm{cherry}, \mathrm{date})$

という文字列ベクトルを作った

Rにおけるベクトルとインデックス

- ベクトルには，インデックス（idx, 要素の番地）が付与されている

例：

```
> x <- c(10, 20, 30, 40, 50)
> x
[1] 10 20 30 40 50
```

idx=1　idx=2　idx=3　idx=4　idx=5

```
> x[2]
[1] 20
```

`x`の2番目の要素を取り出す操作

- Rでは最初の要素のインデックスは `1` であることに注意
  - Python や C では `0` はじまりで形式が異なる
- Rでは単独の数値も長さ1のベクトルとして扱われる

例：

```
> x <- 42
> x[1]
[1] 42
```

1つの数値だけで変数を作ってもインデックスが付与される

ベクトルのインデックス操作の基本

- Rの基本操作 ≒ 配列のインデックス操作

インデックス操作の基本系

#	インデックスの型	例	出力
1	正の整数	`x[2]`	その位置の要素
2	負の整数	`x[-2]`	その位置を除いた要素
3	論理値	`x[c(T,F,...)]`	`TRUE` の位置の要素
4	名前（文字列）	`x[“a”]`	その名前の要素
5	空	`x[]`	全要素（インデックス未指定と同じ）
6	ゼロ	`x[0]`	長さ0の配列（ `numeric(0)` ）

ベクトルのインデックス操作の基本

1. 正の整数インデックス

```
> x <- c(10, 20, 30, 40, 50)
> x[1] ← 1番目の要素
[1] 10
> x[3] ← 3番目の要素
[1] 30
```

複数位置を一度に指定/範囲指定できる

```
> x[c(1,3,5)]
[1] 10 30 50
> x[2:4]
[1] 20 30 40
```

重複・順序の入れ替えも可能

```
> x[c(1,1,2,2,3)]
[1] 10 10 20 20 30
> x[c(3,1,2)]
[1] 30 10 20
```

ベクトルのインデックス操作の基本

2. 負の整数インデックス

```
> x <- c(10, 20, 30, 40, 50)
> x[-1]← 先頭を除く全要素
[1] 20 30 40 50
> x[-c(1,5)] ← 先頭と末尾を除く要素
[1] 20 30 40
> x[-c(2:4)] ← 2~4番目を除く要素
[1] 10 50
```

注意：インデックスの正負は混ぜられない（エラーになる）

```
> x[c(-1, 2)]
Error in x[c(-1, 2)] : only 0's may be mixed
with negative subscripts
```

ベクトルのインデックス操作の基本

3. 論理値インデックス：TRUEの位置の要素だけを取り出す　**最重要インデックス**

```
> x[c(TRUE,FALSE,TRUE,FALSE,TRUE)]
[1] 10 30 50
```

例：比較演算子（論理値ベクトルを返す）

```
> x > 25
[1] FALSE FALSE  TRUE  TRUE  TRUE
# そのままインデックスに使える
> x[x>25]
[1] 30 40 50
```

リサイクル規則：論理ベクトルが短いと繰り返される

```
> x[c(TRUE,FALSE)])
[1] 10 30 50
```

← 長さ2の`x[c(TRUE,FALSE)]`が繰り返されて`x[c(TRUE,FALSE,TRUE,FALSE,TRUE)]`として扱われる

ベクトルのインデックス操作の基本

4. 名前インデックス：要素に名前がついていれば文字列で参照可能

```
> scores <- c(math = 85, physics = 92, chem = 78, bio = 88)
> Scores
   math physics    chem     bio
     85      92      78      88
> scores["physics"]
   physics
        92
> scores[c("math", bio")]
   math  bio
     85   88
```

ベクトルのインデックス操作の基本

5. 空インデックス

```
> x <- c(10, 20, 30)
> x[]              # x と同じ
[1] 10 20 30
```

元の変数との違い:代入操作の挙動

```
> y <- c(1, 2, 3)
> z <- c(1, 2, 3)
> y <- 0          # y は「長さ 1 の 0」になる
> z[] <- 0        # z は「長さ 3 の 0,0,0」になる(中身だけ置換)
> y
[1] 0
> z
[1] 0 0 0
```

論理ベクトルによるインデックス操作

条件を表す論理ベクトルを作ることで様々なインデックス操作が可能になる

1. 比較演算子

演算子	意味
`==`	等しい
`!=`	等しくない
`<` , `<=`	より小さい / 以下
`>` , `>=`	より大きい / 以上

例:

```
> x <- c(10, 20, 30, 40, 50)
> x == 30
[1] FALSE FALSE  TRUE FALSE FALSE
> x != 30
[1]  TRUE  TRUE FALSE  TRUE  TRUE
> x >= 30
[1] FALSE FALSE  TRUE  TRUE  TRUE
> x < 30
[1]  TRUE  TRUE FALSE FALSE FALSE
```

論理ベクトルによるインデックス操作

条件を表す論理ベクトルを作ることで様々なインデックス操作が可能になる

2. 論理演算子

演算子	意味
`&`	要素ごとのAND
`|`	要素ごとのOR
`!`	否定
`&&` , `||`	先頭要素のみのAND/OR

インデックス操作に用いるのは
基本的に `&` と `|` と `!` の3つ

例:

```
# 20 以上 40 以下
> (x >= 20) & (x <= 40)
[1] FALSE  TRUE  TRUE  TRUE FALSE
> x[(x >= 20) & (x <= 40)]
[1] 20 30 40
> # 20 未満 または 40 超
> (x < 20) | (x > 40)
[1]  TRUE FALSE FALSE FALSE  TRUE
> x[(x < 20) | (x > 40)]
[1] 10 50
> # 30 でない
> x[!(x == 30)]# x[x != 30] と同じ
[1] 10 20 40 50
```

論理ベクトルによるインデックス操作

条件を表す論理ベクトルを作ることで様々なインデックス操作が可能になる

3. `%in%` 演算子：集合のメンバシップを論理ベクトルで返す

例：

```
> fruits <- c("apple", "banana", "cherry", "date", "elderberry")
> fruits %in% c("banana", "date")
[1] FALSE  TRUE FALSE  TRUE FALSE
> fruits[fruits %in% c("banana", "date")]
[1] "banana" "date"
> # 一致しないもの
> fruits[!(fruits %in% c("banana", "date"))]
[1] "apple"      "cherry"     "elderberry"
```

複数の値と一致チェックするときに `==` を `|` でつなぐ代わりに使える

位置を返す関数

1. `which()` 関数：インデックス論理ベクトルから TRUE の 位置 を返す

例：

```
> x <- c(10, 50, 30, 20, 40)
> x > 25                          # 論理ベクトル
[1] FALSE  TRUE  TRUE FALSE  TRUE
> which(x > 25)                   # TRUE のインデックス（整数）
[1] 2 3 5
> x[which(x > 25)]               # x[x > 25]  と同じ結果になる
[1] 50 30 40
```

注意：
`x[x > 25]`（論理インデックス）と `x[which(x > 25)]`（整数インデックス）はほぼ同じ結果だが、NA を含むときの挙動が異なる

位置を返す関数

2. `which.max()` , `which.min()` 関数:最大・最小の位置を返す

例:

```
> scores <- c(tanaka = 72, suzuki = 91, sato = 68, yamada = 85)
> which.max(scores)          # 最大値のインデックス（名前付き）
suzuki
     2
> scores[which.max(scores)] # 最大値そのもの
suzuki
    91
> which.min(scores)
sato
   3
> scores[which.min(scores)]
sato
  68
```

位置を返す関数

3. `match()` 関数：最初に一致する位置を返す

例：

```
> fruits <- c("apple", "banana", "cherry", "banana", "apple")
> # "banana" が最初に出現する位置
> match("banana", fruits)
[1] 2
> # 複数のクエリを一度に
> match(c("cherry", "apple", "grape"), fruits)
[1]  3  1 NA
```

"grape" は無いので NAが出力される

インデックスを用いた代入操作

`x[...]` を 左辺 に書くとその位置の値を書き換えられる（多用する機能）

```
> x <- c(10, 20, 30, 40, 50)
> # 1 番目を書き換え
> x[1] <- 99
> x
[1] 99 20 30 40 50
# 複数位置を一度に書き換え（ベクトル代入）
> x[c(2, 4)] <- c(200, 400)
> x
[1]  99 200  30 400  50
```

インデックスを用いた代入操作

例：論理インデックスで条件に合う要素を書き換える

```
> v <- c(3, -1, 4, -1, 5, -9, 2, 6, -5, 3)
> # 負の値を 0 に置き換える
> v[v < 0] <- 0
> v
 [1] 3 0 4 0 5 0 2 6 0 3
```

例：リサイクル（右辺が短いと同じ処理が繰り返される）

```
> w <- c(1, 2, 3, 4, 5, 6)
> w[c(1, 3, 5)] <- 0     # 3 箇所すべて
> w
[1] 0 2 0 4 0 6
> u <- c(1, 2, 3, 4, 5, 6)
> u[c(1, 2, 3, 4)] <- c(-1, -2)  #c(-1,-2)が繰り返される
> u
[1] -1 -2 -1 -2  5  6
```

Chap1. R言語による計算

- インデックス操作
- ベクトル演算
- 行列演算

ベクトルの生成

`c()` 関数，`:` 演算子以外に `seq()` 関数や `rep()` 関数などでベクトル生成が可能

例：`seq()` による等差数列

```
> v2 <- seq(from = 0, to = 1, by = 0.25) # seq()による等差数列
> v2
[1] 0.00 0.25 0.50 0.75 1.00
> v3 <- seq(from = 0, to = 10, length.out = 5) #seq()で長さ指定
> v3
[1]  0.0  2.5  5.0  7.5 10.0
```

ベクトルの生成

`c()` 関数，`:` 演算子以外に `seq()` 関数や `rep()` 関数などでベクトル生成が可能

例：`seq()` による等差数列

初項の値　終項の値　公差

```
> v2 <- seq(from = 0, to = 1, by = 0.25)
> v2
[1] 0.00 0.25 0.50 0.75 1.00
> v3 <- seq(from = 0, to = 10, length.out = 5) #seq()で長さ指定
> v3
[1]  0.0  2.5  5.0  7.5 10.0
```

数列の長さ

ベクトルの生成

`c()` 関数，`:` 演算子以外に `seq()` 関数や `rep()` 関数などでベクトル生成が可能

例：`rep()` で繰り返し

```
> v4 <- rep(c(1, 2), times = 3)
> v4
[1] 1 2 1 2 1 2
> v5 <- rep(c(1, 2), each = 3)
> v5
[1] 1 1 1 2 2 2
```

ベクトルの生成

`c()` 関数，`:` 演算子以外に `seq()` 関数や `rep()` 関数などでベクトル生成が可能

例：`rep()` で繰り返し

繰り返すベクトル　　繰り返し回数

```
> v4 <- rep(c(1, 2), times = 3)
> v4
[1] 1 2 1 2 1 2
> v5 <- rep(c(1, 2), each = 3)
> v5
[1] 1 1 1 2 2 2
```

各要素を何回繰り返すか

オプションの指定の仕方で生成されるベクトルが変わるので注意

ベクトルの属性

Rのベクトルオブジェクトには様々な属性が付与されている

```
> x <- c(10, 20, 30, 40, 50)
> length(x)          # 要素数
[1] 5
> typeof(x)          # 内部型
[1] "double"
> class(x)           # クラス  ← データの型
[1] "numeric"
> is.numeric(x)      # 数値ベクトルか？
[1] TRUE
```

ベクトル演算（要素ごとの演算）

R のベクトル演算は 要素ごと（element-wise）に行われる

```
> a <- c(1, 2, 3, 4)
> b <- c(10, 20, 30, 40)
> a + b                # 加算
[1] 11 22 33 44
> a * b                # 乗算（要素ごと）
[1]  10  40  90 160
> b / a                # 除算
[1] 10 10 10 10
> a^2                  # 冪乗
[1]  1  4  9 16
> sqrt(a)              # 平方根
[1] 1.000000 1.414214 1.732051 2.000000
```

よく使うベクトル関数 1

```
> x <- c(3, 1, 4, 1, 5, 9, 2, 6, 5, 3)
> sum(x)        # 合計
[1] 39
> mean(x)       # 平均
[1] 3.9
> median(x)     # 中央値
[1] 3.5
> var(x)        # 分散（不偏分散、n-1 で割る）
[1] 6.1
> sd(x)         # 標準偏差
[1] 2.469818
> min(x); max(x)
[1] 1
[1] 9
> range(x)      # 最小・最大
[1] 1 9
```

よく使うベクトル関数 2

```
> sort(x)        # 昇順ソート
 [1] 1 1 2 3 3 4 5 5 6 9
> sort(x, decreasing = TRUE)   # 降順ソート
 [1] 9 6 5 5 4 3 3 2 1 1
> rev(x)         # 逆順
 [1] 3 5 6 2 9 5 1 4 1 3
> unique(x)      # 重複除去
[1] 3 1 4 5 9 2 6
> cumsum(x)      # 累積和
 [1]  3  4  8  9 14 23 25 31 36 39
```

よく使うベクトル関数 3：内積（ドット積）

```
> a <- c(1, 2, 3)
> b <- c(4, 5, 6)
> # 方法 1: sum(a * b)
> sum(a * b)
[1] 32
> # 方法 2: %*% 演算子（行列演算と同じ）
> a %*% b            # 結果は 1x1 行列
     [,1]
[1,]   32
> # スカラーに変換するなら
> as.numeric(a %*% b)
[1] 32
```

よく使うベクトル関数 3：内積（ドット積）

```
> a <- c(1, 2, 3)
> b <- c(4, 5, 6)
> # 方法 1: sum(a * b)
> sum(a * b)
[1] 32
> # 方法 2: %*% 演算子（行列演算と同じ）
> a %*% b                # 結果は 1x1 行列
     [,1]
[1,]   32
> # スカラーに変換するなら
> as.numeric(a %*% b)
[1] 32
```

← 要素ごとの掛け算をしてから全要素の総和をとる

← 引数オブジェクトをnumeric型に変換する関数

Chap1. R言語による計算

- インデックス操作
- ベクトル演算
- 行列演算

行列の生成

行列は 2 次元配列で，すべての要素が同じ型である必要がある

```
> A <- matrix(1:12, nrow = 3, ncol = 4)
> A                                  # デフォルトは列方向（byrow = FALSE）
     [,1] [,2] [,3] [,4]
[1,]    1    4    7   10
[2,]    2    5    8   11
[3,]    3    6    9   12
> B <- matrix(1:12, nrow = 3, ncol = 4, byrow = TRUE)
> B                                  # 行方向に詰める
     [,1] [,2] [,3] [,4]
[1,]    1    2    3    4
[2,]    5    6    7    8
[3,]    9   10   11   12
```

行列の生成

行列を生成する関数　行列の要素　行数　列数

```
> A <- matrix(1:12, nrow = 3, ncol = 4)
> A                                  # デフォルトは列方向（byrow = FALSE）
     [,1] [,2] [,3] [,4]
[1,]    1    4    7   10
[2,]    2    5    8   11
[3,]    3    6    9   12
> B <- matrix(1:12, nrow = 3, ncol = 4, byrow = TRUE)
> B                                  # 行方向に詰める
     [,1] [,2] [,3] [,4]
[1,]    1    2    3    4
[2,]    5    6    7    8
[3,]    9   10   11   12
```

行列の生成

特別な行列を生成する関数が用意されている

```
> I3 <- diag(3)                # 3x3 単位行列の生成
> I3
     [,1] [,2] [,3]
[1,]    1    0    0
[2,]    0    1    0
[3,]    0    0    1
> D <- diag(c(1, 2, 3))        # 対角成分を指定（対角行列の生成）
> D
     [,1] [,2] [,3]
[1,]    1    0    0
[2,]    0    2    0
[3,]    0    0    3
```

行列の属性

行列オブジェクトにも様々な属性が付与されている

```
> dim(A)              # 次元（行方向，列方向）
[1] 3 4
> nrow(A); ncol(A)  # 行数・列数
[1] 3
[1] 4
> length(A)           # 全要素数
[1] 12
> dimnames(A)         # 行名・列名（未指定なら NULL）
NULL
```

行列の属性

行名・列名の付与

```
> rownames(A) <- c("r1", "r2", "r3")
> colnames(A) <- c("c1", "c2", "c3", "c4")
> A
   c1 c2 c3 c4
r1  1  4  7 10
r2  2  5  8 11
r3  3  6  9 12
```

行列オブジェクトのインデックス操作

```
> A[2, 3]              # 2 行 3 列目の要素
[1] 8
> A[1, ]               # 1 行目全体（ベクトルになる）
c1 c2 c3 c4
 1  4  7 10
> A[, 2]               # 2 列目全体
r1 r2 r3
 4  5  6
> A[1:2, 3:4]          # 部分行列
   c3 c4
r1  7 10
r2  8 11
> A[, -1]              # 1 列目以外
   c2 c3 c4
r1  4  7 10
r2  5  8 11
r3  6  9 12
```

行列のインデックスは2次元配列になる
1次元目が行方向の操作, 2次元目が列方向の操作を担当する

行列オブジェクトのインデックス操作

```
> # ドロップ防止：部分行列を行列のまま保持
> A[1, , drop = FALSE]
   c1 c2 c3 c4
r1  1  4  7 10
> # 論理インデックス
> A[A > 6]            # 結果はベクトル
[1]  7  8  9 10 11 12
```

行列のインデックスは2次元配列になる
1次元目が行方向の操作, 2次元目が列方向の操作を担当する

行列演算：要素ごとの演算

ベクトルと同様、+ - * / は 要素ごと の演算となる

```
> A <- matrix(1:4, 2, 2)
> B <- matrix(c(10, 20, 30, 40), 2, 2)
> A + B
     [,1] [,2]
[1,]   11   33
[2,]   22   44
> A * B              # 要素ごとの積（アダマール積）
     [,1] [,2]
[1,]   10   90
[2,]   40  160
> A / B
     [,1] [,2]
[1,]  0.1  0.1
[2,]  0.1  0.1
```

行列演算：行列積（%*% 演算子）

行列どうしの通常の積は `%*%` 演算子で行う

```
> A <- matrix(1:6, nrow = 2)          # 2x3行列
> B <- matrix(1:6, nrow = 3)          # 3x2行列
> A %*% B                        # (2x3)%*%(3x2)=2x2行列
     [,1] [,2]
[1,]   22   49
[2,]   28   64
> B %*% A                        # (3x2)%*%(2x3)=3x3行列
     [,1] [,2] [,3]
[1,]    9   19   29
[2,]   12   26   40
[3,]   15   33   51
```

行列演算：転置・逆行列・行列式 1

Rでは種々の行列操作のための関数が予め用意されている

```
> A <- matrix(c(4, 2, 7, 6), nrow = 2)
> A
     [,1] [,2]
[1,]    4    7
[2,]    2    6
> t(A)                # 転置
     [,1] [,2]
[1,]    4    2
[2,]    7    6
> det(A)              # 行列式
[1] 10
```

行列演算：転置・逆行列・行列式 2

Rでは種々の行列操作のための関数が予め用意されている

```
> solve(A)            # 逆行列
     [,1] [,2]
[1,]  0.6 -0.7
[2,] -0.2  0.4
>
> # 検算: A %*% A^(-1) = I
> A %*% solve(A)
              [,1]          [,2]
[1,]  1.000000e+00 -1.110223e-16
[2,] -1.110223e-16  1.000000e+00
```

便利な行列操作：`rbind()` と `cbind()`

`rbind()` と `cbind()` によって2つ以上の行列やベクトルを結合した行列が作れる

```
> r1 <- c(1, 2, 3)
> r2 <- c(4, 5, 6)
> r3 <- c(7, 8, 9)
>
> 列方向に結合していく
> M_row <- rbind(r1, r2, r3)
> M_row
   [,1] [,2] [,3]
r1    1    2    3
r2    4    5    6
r3    7    8    9
```

便利な行列操作：`rbind()` と `cbind()`

`rbind()` と `cbind()` によって2つ以上の行列やベクトルを結合した行列が作れる

```
> r1 <- c(1, 2, 3)
> r2 <- c(4, 5, 6)
> r3 <- c(7, 8, 9)
>
> 行方向に結合していく
> M_col <- cbind(r1, r2, r3)
> M_col
     r1 r2 r3
[1,]  1  4  7
[2,]  2  5  8
[3,]  3  6  9
```

Chap2. 組み込み関数とパッケージ

- 組み込み関数の利用
- 外部パッケージのインストール
- 外部パッケージのインポートと利用

Chap2. 組み込み関数とパッケージ

- 組み込み関数の利用
- 外部パッケージのインストール
- 外部パッケージのインポートと利用

組み込み関数

- Rに最初から含まれている関数で，追加パッケージをインストールせずすぐ使える

カテゴリ	代表的な組み込み関数			
算術演算	`sum()`	`mean()`	`sqrt()`	`abs()`
統計処理	`var()`	`sd()`	`median()`	`quantile()`
データ構造生成	`c()`	`matrix()`	`list()`	`data.frame()`
乱数生成	`rnorm()`	`runif()`	`sample()`	`set.seed()`

- 組み込み関数の一覧を確認する方法：`library(help = "base")`

組み込みの `base` パッケージにある関数の一覧を表示

組み込み関数を使った操作の例

- ベクトルの定義と操作

演習：RStudioで以下のコードを実行してベクトルを定義し，いろいろな操作をしてみよう

```
x <- c(1, 2, 3, 4, 5)
sum(x)
mean(x)
sqrt(x)
sample(x, 3)
```

ベクトル x の定義

x の要素の合計

x の要素の算術平均

x の「各要素」の平方根 ⟶ 要素ごとの演算になる

x の要素の中から3つランダム抽出

- 乱数生成

演習：RStudioで以下のコードを実行していろいろな乱数を生成してみよう

```
runif(5, min = 0, max = 1)
rnorm(5, mean = 100, sd = 15)
rbinom(5, size = 10, prob = 0.5)
```

0～1の範囲で一様乱数を5個生成

平均100，標準偏差15の正規分布に従う乱数を5個生成

成功確率0.5，試行回数10回の二項分布に従う乱数を5個生成

Rにおける乱数生成と乱数シード

- 乱数シードとは？
  乱数生成の出発点となる初期値．これを設定することで同じ乱数列を再現することができる
- 乱数シードの固定方法：`set.seed(数値)`

乱数シードを固定しない

```
> runif(5, min = 0, max = 1)
[1] 0.05352104 0.98155767 0.15084077 0.78914458 0.68673811
> runif(5, min = 0, max = 1)
[1] 0.2685128 0.7562047 0.3970674 0.1572252 0.4856097
> runif(5, min = 0, max = 1)
[1] 0.62152182 0.08209173 0.95623535 0.06640249 0.51158993
>
```

毎回異なる結果が得られる

乱数シードを固定

```
> set.seed(46)
> runif(5, min = 0, max = 1)
[1] 0.1843486 0.2433627 0.5839976 0.3456296 0.2331262
> set.seed(46)
> runif(5, min = 0, max = 1)
[1] 0.1843486 0.2433627 0.5839976 0.3456296 0.2331262
> set.seed(46)
> runif(5, min = 0, max = 1)
[1] 0.1843486 0.2433627 0.5839976 0.3456296 0.2331262
>
```

毎回同じ結果が得られる

なぜ乱数シードが重要か？

- 同じコードを別の人が実行しても結果が一致する
- シミュレーションや検証実験で公平な比較ができる
- 学術論文やレポートで再現可能性（reproducibility）が求められるときに有効

Rにおける乱数生成と乱数シード

- 乱数シードとは？
  乱数生成の出発点となる初期値．これを設定することで同じ乱数列を再現することができる
- 乱数シードの固定方法：`set.seed(数値)`

演習：RStudioで，以下の乱数生成を3回ずつ行い，結果を比較してみよう

- 乱数シードを固定せず，平均100，標準偏差15の正規分布に従う乱数を5個生成
- 乱数シードを固定して，平均100，標準偏差15の正規分布に従う乱数を5個生成

☕ **演習**：Rの乱数生成 💻

☕Rによる数値計算とベクトル演算

演習：Rのベクトル演算 💻

- Rでは，ベクトル演算を活用することで繰り返し処理を大幅に効率化できる（むしろこれを使わないと非常に処理が遅い）

例：1千万要素の和の計算

ベクトル演算を使用

```
# ベクトルを準備
x <- runif(1e7)
y <- runif(1e7)

# 2. ベクトル演算による加算
system.time({
  result_vec <- x + y
})
```

計算時間（colab）：0.164 (sec)

forループで計算

```
system.time({
  result_loop <- numeric(1e7)  # 初期化
  for (i in 1:n) {
    result_loop[i] <- x[i] + y[i]
  }
})
```

計算時間（colab）：1.492 (sec)

Chap2. 組み込み関数とパッケージ

- 組み込み関数の利用
- 外部パッケージのインストール
- 外部パッケージのインポートと利用

外部パッケージのインストール

- 外部パッケージをインストールする方法：
  コンソールで以下を実行

```
install.packages("パッケージ名", repos="ミラーサイトのURL")
```

- 例えば `ggplot2` パッケージをインストールしたい場合を実行

```
install.packages("ggplot2", repos="https://cloud.r-project.org")
```

RStudio社提供のグローバルミラーで非常に安定・高速

- インストールが完了したら `library(パッケージ名)` でインポートできる

```
library(ggplot2)
```

← library() で呼び出すときは " " は不要な点に注意

演習：RStudioで上記のコードを実行して `ggplot2` をインストールしてみよう

インストールした ggplot2 を使ってみよう

演習：RStudioで下記のコードを実行して ggplot2 を使ってみよう

1. ggplot2 のインポート：library(ggplot2) を実行
2. データの準備：以下を実行

```
my_data <- data.frame(
  x = c(1, 2, 3, 4, 5),
  y = c(2, 4, 1, 8, 7)
)
```

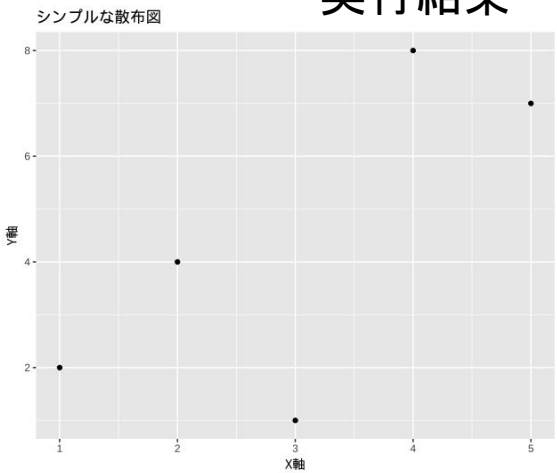
3. 散布図を描く：以下を実行

```
①ggplot(data = my_data,aes(x = x, y = y)) + ②geom_point()
   + ③labs(title = "シンプルな散布図", x = "X軸", y = "Y軸")
```

①：データ＋プロットの美術（aesthetics）を指定
②：プロットの幾何オブジェクト（geom）
③：オプションの指定

これらを + 演算子で繋げていくだけ

実行結果

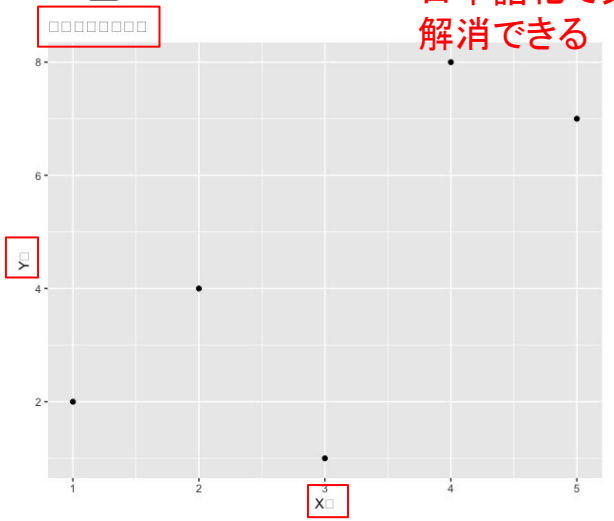


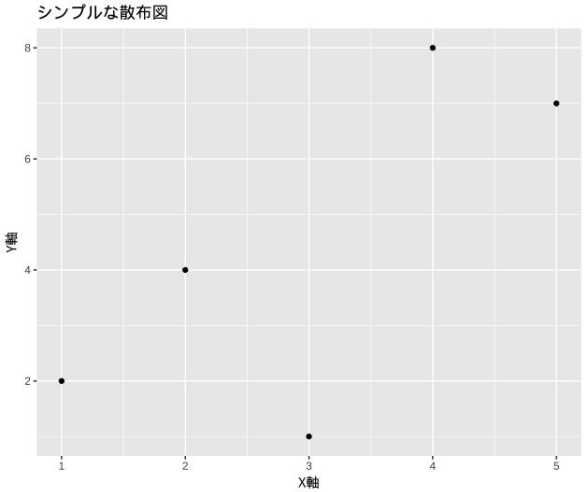
インストールした ggplot2 を使ってみよう(補足)

- プロットが文字化けするとき
  → 以下を実行して日本語フォントをインポート・有効化する

```
install.packages("showtext", repos="https://cloud.r-project.org")
library(showtext)
showtext_auto()
```

日本語化で文字化けを
解消できる





☕ Rによる乱数生成とプロット

演習：RStudioで実行してみよう

1. 平均が50，分散が10の正規分布から乱数を1000個生成し，ヒストグラムとしてプロットしてみよう

   ヒント

   - 正規分布に従う乱数の生成は `rnorm()` 関数で
   - `ggplot2` でヒストグラムを描画するためには `geom_histogram()` 関数を使う

2. 乱数シードを変えて実行し，ヒストグラムがどう変わるか見てみよう

* サンプルコードは講義後に公開します

提出課題

ggplot2 で作成したプロットファイル（プロットの内容は問わない）を提出
ただし，タイトルや軸ラベルなどに文字化けがないように作成すること

提出方法：KULMSの課題提出機能
提出期限：4/22（水）23:55